Programming and Numerical Methods for Economics (ECNM10115)

University of Edinburgh

Mock Examination — Academic Year 2026/27

— Model Solutions —

Date: Practice Paper
Duration: 2 hours
Total marks: 50
Equipment: Word processor only (no Python environment)

Instructions.

- Answer **all** questions and sub-questions. Type your answers in the word processor provided. This is a closed-book examination; no external materials, websites, or AI tools may be consulted.
- When asked to write code, type well-formatted Python code. You will be assessed on correctness of logic and syntax; you do *not* need to run the code.
- When asked for explanations, write concise prose (2–4 sentences unless stated otherwise). Any mathematical derivations or proofs should be written in complete sentences with clear reasoning.
- **Partial credit will be awarded.** If you get stuck on a problem and are unable to complete it, write a short and clear description of where you got stuck, what you think is going wrong, and how you would attempt to solve the remainder. A careful and thoughtful attempt, even if unsuccessful or unable to produce working code, will be rewarded. For each question, try to demonstrate what you have learned during the course.
- You may assume that `numpy`, `pandas`, `scipy`, and `matplotlib` are imported as usual:

```
import numpy as np
import pandas as pd
from scipy.optimize import fsolve, minimize
import matplotlib.pyplot as plt
```

Question 1 — Programming Fundamentals and Debugging [7 marks]

Part (a) [3 marks]

The following function is intended to compute the **steady-state capital stock** of a representative-agent economy with log utility, Cobb–Douglas production, and partial depreciation. The steady state satisfies the Euler equation $1 = \beta\big(\alpha A\, k^{\alpha-1} + 1 - \delta\big)$, and the function should return the value of k^*.

The code below contains **three errors**. For each error, (i) state the line number, (ii) explain what is wrong, and (iii) write the corrected line.

```
1 def steady_state_capital(alpha, beta, delta, A):
2     """Return the steady-state capital stock k*."""
3     r_star = 1 / beta - 1 + delta
4     k_star = (r_star / (alpha * A))**(1 / (1 - alpha))
5     y_star = A * k_star**A
6     c_star = y_star - delta * k_star**2
7     return k_star, y_star, c_star
```

Solution

Error 1 — Line 4: The exponent should be $1/(\alpha-1)$, not $1/(1-\alpha)$. From the Euler equation at steady state: $\alpha A\, k^{\alpha-1} = r^* = 1/\beta - 1 + \delta$, so $k^{\alpha-1} = r^*/(\alpha A)$ and $k^* = (r^*/(\alpha A))^{1/(\alpha-1)}$.
Corrected line 4: `k_star = (r_star / (alpha * A))**(1 / (alpha - 1))`

Error 2 — Line 5: The exponent on k is `A` (the TFP parameter) instead of `alpha`. Output is $y = A\, k^{\alpha}$.
Corrected line 5: `y_star = A * k_star**alpha`

Error 3 — Line 6: The consumption formula is wrong in two ways: it uses k^2 instead of k, and the resource constraint in steady state is $c^* = y^* - \delta\, k^*$ (investment equals depreciation).
Corrected line 6: `c_star = y_star - delta * k_star`

Part (b) [4 marks]

Consider the CRRA (Constant Relative Risk Aversion) utility function used in heterogeneous-agent models:

$$u(c) = \frac{c^{1-\sigma}}{1-\sigma}, \qquad \sigma > 0,\ \sigma \neq 1.$$

When $\sigma = 1$, the utility function reduces to $u(c) = \ln(c)$.

(i) Write a Python function `crra_utility(c, sigma)` that computes the CRRA utility, handling both the general case and the special case $\sigma = 1$. Include a brief docstring.
(ii) Derive the marginal utility $u'(c)$ for the general case ($\sigma \neq 1$) and verify that it equals $c^{-\sigma}$.
(iii) Explain in 2–3 sentences why a higher σ makes the agent more risk-averse, referring to the concavity of u.

Solution

(i)

```python
def crra_utility(c, sigma):
    """Compute CRRA utility. Falls back to log when sigma = 1."""
    if sigma == 1:
        return np.log(c)
    else:
        return c**(1 - sigma) / (1 - sigma)
```

(ii) Differentiating with respect to c:

$$u'(c) = \frac{d}{dc}\left[\frac{c^{1-\sigma}}{1-\sigma}\right] = \frac{(1-\sigma)\, c^{-\sigma}}{1-\sigma} = c^{-\sigma}.$$

The $(1-\sigma)$ factors cancel, confirming $u'(c) = c^{-\sigma}$.

(iii) The second derivative is $u''(c) = -\sigma\, c^{-\sigma-1} < 0$, so the function is concave. The coefficient of relative risk aversion is $-c\, u''(c)/u'(c) = \sigma$, which is constant and increasing in σ. A higher σ means stronger concavity (the marginal utility of consumption drops faster), so the agent is more averse to consumption fluctuations and prefers smoother consumption paths.

Question 2 — Stochastic Productivity and Markov Chains [9 marks]

Part (a) — AR(1) process for TFP [4 marks]

In the stochastic growth model, the log of total factor productivity follows an AR(1) process:

$$\log(z_{t+1}) = \rho \log(z_t) + \sigma_\varepsilon \varepsilon_{t+1}, \qquad \varepsilon_t \sim N(0,1).$$

Assume $\rho = 0.90$, $\sigma_\varepsilon = 0.04$.

(i) Write a Python function `simulate_tfp(rho, sigma_e, T, z0=1.0)` that simulates the *level* of productivity z_t (not its log) for T periods, starting from z_0. Return a NumPy array of length $T+1$.

(ii) Derive the unconditional variance of $\log(z_t)$ when $|\rho| < 1$.

(iii) Compute the unconditional standard deviation of $\log(z_t)$ for the given parameters. Explain what this tells us about the magnitude of TFP fluctuations in the model.

(iv) What happens to the variance of $\log(z_t)$ as $\rho \to 1$? Relate this to the concept of a "unit root" in macroeconometrics.

Solution

(i)

```
def simulate_tfp(rho, sigma_e, T, z0=1.0):
    log_z = np.zeros(T + 1)
    log_z[0] = np.log(z0)
    eps = np.random.normal(0, 1, T)
    for t in range(T):
        log_z[t + 1] = rho * log_z[t] + sigma_e * eps[t]
    return np.exp(log_z)
```

(ii) Let $x_t = \log(z_t)$. Then $x_{t+1} = \rho x_t + \sigma_\varepsilon \varepsilon_{t+1}$. In the stationary regime, $\text{Var}(x_t) = \text{Var}(x_{t+1}) \equiv V$:

$$V = \rho^2 V + \sigma_\varepsilon^2 \quad \Longrightarrow \quad V(1-\rho^2) = \sigma_\varepsilon^2 \quad \Longrightarrow \quad \boxed{\text{Var}(\log z_\infty) = \frac{\sigma_\varepsilon^2}{1-\rho^2}}.$$

(iii) $\text{SD}(\log z_\infty) = \sigma_\varepsilon / \sqrt{1-\rho^2} = 0.04/\sqrt{1-0.81} = 0.04/\sqrt{0.19} \approx 0.04/0.4359 \approx 0.0918$. This means that in the long run, $\log(z_t)$ fluctuates with a standard deviation of about 9.2% around its mean of zero. In levels, this translates to TFP varying roughly $\pm 9\%$ from its mean, generating the business-cycle fluctuations that the model aims to capture.

(iv) As $\rho \to 1$, the denominator $(1-\rho^2) \to 0$, so $\text{Var}(\log z_t) \to \infty$. Log productivity becomes a random walk ($\log z_{t+1} = \log z_t + \sigma_\varepsilon \varepsilon_{t+1}$) with variance growing linearly in time ($t\,\sigma_\varepsilon^2$). This is a "unit root" process: shocks have permanent effects and no stationary distribution exists, which has important implications for whether macroeconomic time series are trend-stationary or difference-stationary.

Part (b) — Discretising the AR(1) [3 marks]

To solve the model numerically, we approximate the continuous AR(1) process with a finite-state Markov chain. Suppose we use $N_z = 3$ grid points and the Rouwenhorst method produces the state space $\{z_1, z_2, z_3\} = \{0.923,\ 1.000,\ 1.083\}$ and the transition matrix:

$$P = \begin{pmatrix} 0.810 & 0.180 & 0.010 \\ 0.090 & 0.820 & 0.090 \\ 0.010 & 0.180 & 0.810 \end{pmatrix}.$$

(i) Compute the stationary distribution $\boldsymbol{\pi}$ by solving $\boldsymbol{\pi} P = \boldsymbol{\pi}$ with $\sum_i \pi_i = 1$. (*Hint*: by the symmetry of P, is $\boldsymbol{\pi}$ uniform?)
(ii) Using $\boldsymbol{\pi}$, compute the unconditional mean of z: $E[z] = \sum_i \pi_i \, z_i$.

Solution

(i) The matrix P is symmetric: $P_{ij} = P_{(N_z+1-i)(N_z+1-j)}$, and the row sums are all 1. By the symmetry of the Rouwenhorst construction, the stationary distribution is:

$$\boxed{\boldsymbol{\pi} = \left(\tfrac{1}{4},\ \tfrac{1}{2},\ \tfrac{1}{4}\right) = (0.25,\ 0.50,\ 0.25)}.$$

We can verify: $\pi P = (0.25 \times 0.810 + 0.50 \times 0.090 + 0.25 \times 0.010, \ldots) = (0.2025 + 0.045 + 0.0025, \ldots) = (0.25, \ldots)$. ✓

(ii)

$$E[z] = 0.25 \times 0.923 + 0.50 \times 1.000 + 0.25 \times 1.083 = 0.2308 + 0.5000 + 0.2708 = \boxed{1.0015}.$$

This is very close to 1, which is the unconditional mean of $z_t = e^{x_t}$ when x_t has mean zero (the exact value is $e^{\sigma_x^2/2} \approx 1.0042$, so the 3-point approximation is reasonably accurate).

Part (c) — Expected output [2 marks]

Using the Cobb–Douglas production function $Y = z\,K^{\alpha}$ with $\alpha = 0.36$ and $K = 5$:

(i) Compute the expected output $E[Y] = E[z] \cdot K^{\alpha}$ using the unconditional mean from Part (b).
(ii) Explain in 2–3 sentences why discretising the AR(1) process is necessary for solving dynamic programming problems numerically.

Solution

(i)

$$E[Y] = E[z] \cdot K^{\alpha} = 1.0015 \times 5^{0.36} = 1.0015 \times 1.8320 \approx \boxed{1.835}.$$

(ii) Value function iteration requires computing conditional expectations of the form $E[V(K', z') \mid z]$, which involves integrating over the continuous distribution of z'. By discretising onto a finite grid with known transition probabilities, the expectation becomes a finite weighted sum $\sum_{j=1}^{N_z} P_{ij}\, V(K', z_j)$, which is straightforward to compute. This makes the dynamic programming problem tractable on a computer while preserving the key statistical properties (persistence, variance) of the original AR(1) process.

Question 3 — Monte Carlo and Numerical Integration [8 marks]

Part (a) — Gauss–Hermite quadrature [3 marks]

When solving dynamic models with continuous shocks $\varepsilon \sim N(0,1)$, we need to compute expectations of the form $E[f(\varepsilon)]$. Gauss–Hermite quadrature approximates this integral using a weighted sum of n nodes:

$$E[f(\varepsilon)] = \int_{-\infty}^{\infty} f(x)\,\frac{1}{\sqrt{2\pi}} e^{-x^2/2}\,dx \;\approx\; \sum_{i=1}^{n} w_i\, f(x_i)\,,$$

where $\{x_i, w_i\}_{i=1}^{n}$ are the quadrature nodes and weights.

(i) Using the NumPy function `np.polynomial.hermite.hermgauss(n)`, write Python code that obtains 5 quadrature nodes and weights suitable for integrating against the standard normal density. (*Hint*: the raw Gauss–Hermite nodes and weights integrate against e^{-x^2}; you need to rescale by $\sqrt{2}$ for the nodes and $1/\sqrt{\pi}$ for the weights.)

(ii) Use your quadrature rule to approximate $E[e^{\varepsilon}]$ with $\varepsilon \sim N(0,1)$. What is the exact value?

(iii) Compared to Monte Carlo simulation, state one advantage and one disadvantage of Gauss–Hermite quadrature.

Solution

(i)

```
n = 5
x_raw, w_raw = np.polynomial.hermite.hermgauss(n)
# Rescale for standard normal integration
x_nodes = x_raw * np.sqrt(2)
w_nodes = w_raw / np.sqrt(np.pi)
# Verify weights sum to 1
print("Sum of weights:", np.sum(w_nodes))  # should be ~1.0
```

(ii)

```
E_exp = np.sum(w_nodes * np.exp(x_nodes))
# E_exp is approximately 1.6487
```

The exact value is $E[e^{\varepsilon}] = M(1) = e^{1/2} = \sqrt{e} \approx 1.6487$, using the moment generating function of $N(0,1)$. Gauss–Hermite with just 5 nodes gives essentially the exact answer.

(iii) Advantage: Gauss–Hermite quadrature is deterministic and extremely accurate for smooth integrands — with just 5–7 nodes it can match or exceed the accuracy of millions of Monte Carlo draws, and it produces the same result every time (no simulation noise).
Disadvantage: It suffers from the "curse of dimensionality" — with d random variables each needing n nodes, the total number of evaluation points is n^d, which quickly becomes infeasible for high-dimensional problems. Monte Carlo, by contrast, converges at rate $1/\sqrt{N}$ regardless of dimension.

Part (b) — Monte Carlo estimation of expected output [3 marks]

Consider an economy where output is $Y = z\,K^{\alpha}$ with $\alpha = 0.36$, $K = 5$, and $\log(z) \sim N(0, \sigma_z^2)$ with $\sigma_z = 0.09$.

(i) Write a Python function `mc_expected_output(n, alpha, K, sigma_z)` that estimates $E[Y]$ by drawing n samples of z and averaging $z\,K^{\alpha}$.

(ii) The exact expected output is $E[Y] = e^{\sigma_z^2/2} \cdot K^{\alpha}$. Compute this value numerically.

(iii) If you run your Monte Carlo with $n = 10{,}000$ and obtain $\hat{E}[Y] = 1.841$, compute the absolute error relative to the exact value.

Solution

(i)

```
def mc_expected_output(n, alpha=0.36, K=5, sigma_z=0.09):
    log_z = np.random.normal(0, sigma_z, n)
    z = np.exp(log_z)
    Y = z * K**alpha
    return np.mean(Y)
```

(ii)

$$E[Y] = e^{\sigma_z^2/2} \cdot K^{\alpha} = e^{0.0081/2} \cdot 5^{0.36} = e^{0.00405} \times 1.8320 = 1.00406 \times 1.8320 \approx \boxed{1.8394}.$$

(iii) Absolute error $= |1.841 - 1.8394| = 0.0016$. Relative error $= 0.0016/1.8394 \approx 0.087\%$. This is very small, illustrating that 10,000 draws already gives a good approximation.

Part (c) — The Gini coefficient [2 marks]

The Gini coefficient measures inequality in a distribution. For a sample of n sorted values $x_1 \leq x_2 \leq \cdots \leq x_n$, it can be computed as:

$$G = \frac{2\sum_{i=1}^{n} i\, x_i}{n\sum_{i=1}^{n} x_i} - \frac{n+1}{n}.$$

(i) Write a Python function `gini(x)` that computes the Gini coefficient from an unsorted array `x`.

(ii) If a simulated economy produces wealth values $\{1, 2, 3, 4, 10\}$, compute the Gini coefficient by hand.

Solution

(i)

```
def gini(x):
    xs = np.sort(x)
    n = len(xs)
    numerator = 2 * np.sum(np.arange(1, n + 1) * xs)
    denominator = n * np.sum(xs)
    return numerator / denominator - (n + 1) / n
```

(ii) Sorted values: $x = \{1, 2, 3, 4, 10\}$, $n = 5$.

$$\sum_{i=1}^{5} i\, x_i = 1(1) + 2(2) + 3(3) + 4(4) + 5(10) = 1 + 4 + 9 + 16 + 50 = 80.$$

$$\sum_{i=1}^{5} x_i = 1 + 2 + 3 + 4 + 10 = 20.$$

$$G = \frac{2 \times 80}{5 \times 20} - \frac{6}{5} = \frac{160}{100} - 1.2 = 1.6 - 1.2 = \boxed{0.40}.$$

Question 4 — Root-Finding and Steady-State Analysis [9 marks]

Part (a) — Analytical steady state [3 marks]

Consider the neoclassical growth model with the Euler equation:

$$\frac{1}{C_t} = \beta\left(\alpha A\, K_{t+1}^{\alpha-1} + 1 - \delta\right)\frac{1}{C_{t+1}},$$

and the resource constraint $C_t + K_{t+1} = A\,K_t^{\alpha} + (1-\delta)\,K_t$.

Use parameters: $\alpha = 0.35$, $\beta = 0.97$, $\delta = 0.06$, and target steady-state output $y^* = 10$.

(i) Derive the steady-state capital stock k^* as a function of α, β, and δ (eliminate A and C). Show your working.

(ii) Write a Python function that uses `fsolve` to find the value of A such that steady-state output $y^* = A\,(k^*)^{\alpha} = 10$.

(iii) Using your solution, compute the steady-state consumption $c^* = A\,(k^*)^{\alpha} - \delta\,k^*$.

Solution

(i) In steady state, $C_t = C_{t+1} = c^*$ and $K_t = K_{t+1} = k^*$. From the Euler equation:

$$1 = \beta\big(\alpha A\, k^{*\,\alpha-1} + 1 - \delta\big) \quad\Longrightarrow\quad \alpha A\, k^{*\,\alpha-1} = \frac{1}{\beta} - 1 + \delta\,.$$

Since $y^* = A\,k^{*\alpha}$, dividing by k^*: $\alpha\, y^*/k^* = 1/\beta - 1 + \delta$, giving:

$$\boxed{k^* = \left(\frac{\alpha A}{1/\beta - 1 + \delta}\right)^{\frac{1}{1-\alpha}}}.$$

(ii)

```
alpha, beta, delta = 0.35, 0.97, 0.06

def steady_state_residual(A):
    k_star = (alpha * A / (1/beta - 1 + delta))**(1/(1-alpha))
    y_star = A * k_star**alpha
    return y_star - 10

A_star = fsolve(steady_state_residual, 3.0)[0]
# Compute k* and y* at solution
k_star = (alpha * A_star / (1/beta - 1 + delta))**(1/(1-alpha))
y_star = A_star * k_star**alpha
```

(iii) The steady-state investment exactly replaces depreciated capital, so:

$$c^* = y^* - \delta\,k^* = A\,(k^*)^{\alpha} - \delta\,k^*\,.$$

From (ii): $1/\beta - 1 + \delta = 1/0.97 - 1 + 0.06 \approx 0.0309 + 0.06 = 0.0909$. We have $y^* = 10$ by construction. Then $k^* = \alpha\,y^*/(1/\beta - 1 + \delta) = 0.35 \times 10/0.0909 \approx 38.50$. So:

$$c^* = 10 - 0.06 \times 38.50 = 10 - 2.31 = \boxed{7.69}.$$

Part (b) — Transition dynamics [3 marks]

Suppose the economy is in steady state and the TFP parameter A increases permanently by 15%. The economy must transition to a new steady state.

(i) Explain why the Euler equation can be rewritten purely in terms of capital:

$$\frac{1}{A_{\text{new}}\, k_t^{\alpha} + (1-\delta)k_t - k_{t+1}} = \beta\, \frac{\alpha A_{\text{new}}\, k_{t+1}^{\alpha-1} + 1 - \delta}{A_{\text{new}}\, k_{t+1}^{\alpha} + (1-\delta)k_{t+1} - k_{t+2}}\,.$$

(ii) Using this equation, explain (in 3–4 sentences) how you would set up a system of $n-2$ nonlinear equations in $n-2$ unknowns $(k_1, k_2, \ldots, k_{n-2})$ to compute the transition path, given k_0 (old steady state) and $k_{n-1} = k_n$ (new steady state).

(iii) Name the `scipy` function you would use to solve this system.

Solution

(i) The resource constraint gives $C_t = A\, k_t^{\alpha} + (1-\delta)k_t - k_{t+1}$, so consumption can be expressed entirely in terms of capital at dates t and $t{+}1$. Substituting this expression for C_t and C_{t+1} into the Euler equation $1/C_t = \beta(\alpha A\, k_{t+1}^{\alpha-1} + 1 - \delta)/C_{t+1}$ eliminates consumption, leaving an equation in k_t, k_{t+1}, and k_{t+2} only.

(ii) We choose n large enough that the economy reaches the new steady state (e.g. $n = 40$). The boundary conditions are $k_0 = k^*_{\text{old}}$ (known) and $k_{n-1} = k_n = k^*_{\text{new}}$ (the new steady state, also known). For each interior period $t = 0, 1, \ldots, n{-}3$, the Euler equation written in terms of (k_t, k_{t+1}, k_{t+2}) gives one equation. This produces $n-2$ equations in the $n-2$ unknowns $k_1, \ldots, k_{n-2}$, which can be stacked into a vector and solved simultaneously.

(iii) `scipy.optimize.fsolve` (or equivalently `scipy.optimize.root`).

Part (c) — Bisection method [3 marks]

The bisection method finds a root of $f(x) = 0$ on an interval $[a, b]$ where $f(a)$ and $f(b)$ have opposite signs.

(i) Write a Python function `bisection(f, a, b, tol=1e-6, max_iter=100)` that implements the bisection method. Return the approximate root.

(ii) Starting from $[a, b] = [0, 2]$, how many iterations are needed to achieve a tolerance of 10^{-6}? (*Hint*: after n iterations the interval width is $(b-a)/2^n$.)

(iii) Give one advantage of bisection over Newton's method and one disadvantage.

Solution

(i)

```
def bisection(f, a, b, tol=1e-6, max_iter=100):
    for i in range(max_iter):
        mid = (a + b) / 2
        if abs(f(mid)) < tol or (b - a) / 2 < tol:
            return mid
        if f(a) * f(mid) < 0:
            b = mid
        else:
            a = mid
    return (a + b) / 2
```

(ii) We need $(b-a)/2^n < 10^{-6}$, i.e. $2/2^n < 10^{-6}$, so $2^n > 2 \times 10^6$. Taking $\log_2$: $n > \log_2(2 \times 10^6) = 1 + 6\log_2(10) \approx 1 + 19.93 = 20.93$. So $\boxed{n = 21}$ iterations suffice.

(iii) Advantage: Bisection is guaranteed to converge (it only requires a sign change, no derivatives), making it very robust even for non-smooth or poorly-behaved functions.
Disadvantage: Bisection converges linearly (one bit of accuracy per iteration), whereas Newton's method converges quadratically (the number of correct digits roughly doubles each iteration), so Newton is much faster when it converges.

Question 5 — Optimisation and Budget Constraints [9 marks]

Part (a) — Consumption–saving decision [4 marks]

Consider a worker who allocates time between work and training. The worker chooses consumption c, savings a', and training time $s \in [0, 1]$. The remaining fraction $1 - s$ of time is devoted to work. The budget constraint is:

$$c + a' = w\, z\, h\, (1 - s) + (1 + r)\, a\,,$$

where w is the wage, z is productivity, h is human capital, r is the interest rate, and a is current assets.

The worker maximises CRRA utility $u(c) = \frac{c^{1-\sigma}}{1-\sigma}$.

(i) Set up the Lagrangian $\mathcal{L}$ for the static problem of choosing c and a' (treating s as given) to maximise $u(c)$ subject to the budget constraint.

(ii) Derive the first-order condition with respect to c. What is the economic interpretation?

(iii) For parameters $w = 1$, $z = 1$, $h = 1.5$, $s = 0.4$, $r = 0.03$, $a = 5$, compute the total cash-on-hand (right-hand side of the budget constraint).

Solution

(i) The Lagrangian is:

$$\mathcal{L} = \frac{c^{1-\sigma}}{1-\sigma} - \lambda\big(c + a' - w\, z\, h\, (1 - s) - (1 + r)\, a\big)\,.$$

(ii) The FOC with respect to c:

$$\frac{\partial \mathcal{L}}{\partial c} = c^{-\sigma} - \lambda = 0 \quad \Longrightarrow \quad \lambda = c^{-\sigma} = u'(c)\,.$$

The Lagrange multiplier λ equals the marginal utility of consumption, which is also the shadow price of the budget constraint — it measures the utility gain from one extra unit of income.

(iii) Cash-on-hand:

$$\text{COH} = w\, z\, h\, (1 - s) + (1 + r)\, a = 1 \times 1 \times 1.5 \times 0.6 + 1.03 \times 5 = 0.9 + 5.15 = \boxed{6.05}\,.$$

Part (b) — Numerical solution with SLSQP [3 marks]

Write complete Python code to solve the following problem numerically using `scipy.optimize.minimize` with `method='SLSQP'`:

$$\max_{c,\, a'} \ \frac{c^{1-\sigma}}{1-\sigma} \qquad \text{s.t.} \quad c + a' = \text{COH}, \quad c > 0, \quad a' \geq 0\,.$$

Use $\sigma = 2$ and $\text{COH} = 6.05$. Your code should include the objective (negated), the budget constraint as an equality constraint, bounds, and the `minimize` call.

Solution

```
sigma = 2
COH = 6.05

def neg_utility(x):
    c, a_prime = x
    return -(c**(1 - sigma) / (1 - sigma))

def budget(x):
    return COH - x[0] - x[1]   # = 0 at feasibility

x0 = [3.0, 3.0]
bounds = [(0.01, None), (0.0, None)]
constraints = {'type': 'eq', 'fun': budget}

result = minimize(neg_utility, x0, method='SLSQP',
                  bounds=bounds, constraints=constraints)
c_star, a_prime_star = result.x
```

Note: Since this is a single-period problem with no future value of savings, the optimal solution sets $a' = 0$ and $c = \text{COH} = 6.05$ (all income is consumed). In a dynamic setting, the continuation value of a' would create an interior solution.

Part (c) — Government budget balance [2 marks]

A government levies a flat tax τ on all income (labour and capital) and uses the revenue to subsidise training at rate γ per unit of effective human capital invested. The government budget constraint is:

$$\tau \int \left[w\, z\, h\, (1-s) + r\, a \right] d\Phi(a,h,z) \;=\; \gamma \int s\, h \; d\Phi(a,h,z)\,,$$

where Φ is the stationary distribution of agents over states (a, h, z).

(i) Explain in 2–3 sentences what this equation says economically.

(ii) If aggregate taxable income is 2.5 and aggregate training investment $(s \cdot h)$ is 0.8, and $\gamma = 0.20$, what tax rate τ balances the budget?

Solution

(i) The equation states that the government's total tax revenue (left-hand side) must equal its total expenditure on training subsidies (right-hand side). The tax revenue is collected as a fraction τ of every agent's total income (wages plus capital income), aggregated over the entire population. The subsidy expenditure pays γ per unit of human-capital-weighted training time, aggregated over all agents who invest in training.

(ii) Setting revenue equal to expenditure:

$$\tau \times 2.5 = 0.20 \times 0.8 = 0.16 \quad \Longrightarrow \quad \tau = \frac{0.16}{2.5} = \boxed{0.064}\,.$$

The required tax rate is 6.4%.

Question 6 — Dynamic Programming and Value Function Iteration [8 marks]

This question considers the numerical solution of a dynamic economic model using **value function iteration** (VFI).

A household with assets a, human capital h, and productivity z solves:

$$V(a, h, z) = \max_{c,\, s,\, a'} \; u(c) + \beta\, E\big[V(a', h', z') \mid z\big]$$

subject to:

$$\begin{aligned} c + a' &= w\, z\, h\,(1 - s) + (1 + r)\, a && \text{(budget)} \\ h' &= (1 - \delta_h)\, h + A_h (s\, h)^{\alpha} && \text{(human capital law of motion)} \\ \log(z') &= \rho\, \log(z) + \sigma_\varepsilon\, \varepsilon', \qquad \varepsilon' \sim N(0,1) && \text{(productivity)} \\ a' &\geq 0, \quad 0 \leq s \leq 1 && \text{(constraints)} \end{aligned}$$

Part (a) — Model structure [3 marks]

(i) Identify the **state variables**, the **control variables** (choice variables), and the **exogenous shock** in this model.

(ii) The state space is 3-dimensional: (a, h, z). If we use grids with $N_a = 50$ points for assets, $N_h = 25$ for human capital, and $N_z = 7$ for productivity, how many grid points does the value function V have?

(iii) Explain in 2–3 sentences why the choice of s (time allocated to training) creates a trade-off for the agent.

Solution

(i)

- **State variables:** assets a, human capital h, productivity z.
- **Control variables:** consumption c, training time s, next-period assets a'.
- **Exogenous shock:** productivity innovation $\varepsilon' \sim N(0,1)$, which drives the AR(1) process for z.

(ii) Total grid points: $N_a \times N_h \times N_z = 50 \times 25 \times 7 = \boxed{8{,}750}$.

(iii) Increasing s (training time) reduces current labour income because the worker devotes less time to working ($1 - s$ decreases), which lowers current consumption. However, more training increases future human capital h' through the accumulation function $h' = (1 - \delta_h)h + A_h(sh)^{\alpha}$, which raises future earnings. The agent must balance the immediate cost of lost income against the long-run benefit of higher productivity.

Part (b) — The VFI algorithm [3 marks]

(i) Describe the value function iteration algorithm in 4–5 numbered steps, from initialisation to convergence. Be specific about what happens at each step.

(ii) The convergence check uses $\max_{a,h,z} |V^{(n+1)}(a, h, z) - V^{(n)}(a, h, z)| < \texttt{tol}$. Explain why this is a valid stopping criterion, referring to the contraction mapping theorem.

(iii) What role does the discount factor $\beta < 1$ play in ensuring convergence?

Solution

(i)

1. **Initialise:** Set an initial guess for the value function $V^{(0)}(a, h, z)$ at every grid point (e.g. $V^{(0)} = 0$ everywhere). Compute the expected value $EV^{(0)}(a, h, z) = \sum_j P(z' = z_j|z)\, V^{(0)}(a, h, z_j)$ using the transition matrix.
2. **Maximise:** For each grid point (a_i, h_j, z_k), solve $\max_{c,s,a'}\ u(c)+\beta\, EV^{(0)}(a', h', z_k)$ subject to the constraints. Store the maximised value as $V^{(1)}(a_i, h_j, z_k)$ and the optimal choices as policy functions.
3. **Update:** Recompute $EV^{(1)}$ from the new value function using the transition matrix.
4. **Check convergence:** Compute $\texttt{err} = \max|V^{(1)} - V^{(0)}|$. If $\texttt{err} < \texttt{tol}$, stop. Otherwise, set $V^{(0)} \leftarrow V^{(1)}$ and return to Step 2.
5. **Output:** Return the converged value function and policy functions (c^*, s^*, a'^*).

(ii) By Blackwell's sufficient conditions, the Bellman operator T is a contraction mapping (with modulus β) on the space of bounded continuous functions. The contraction mapping theorem guarantees that T has a unique fixed point V^* and that $\|T^n V^{(0)} - V^*\|_\infty \to 0$. The sup-norm error $\max|V^{(n+1)} - V^{(n)}|$ bounds the distance to the true solution, so once it is below the tolerance, we are close to V^*.

(iii) The discount factor $\beta < 1$ ensures that the Bellman operator is a contraction with modulus β. If $\beta \geq 1$, the operator would not be a contraction, the value function could diverge (infinite lifetime utility), and VFI would not converge. Economically, $\beta < 1$ means agents discount the future, which bounds lifetime utility and guarantees a well-defined solution.

Part (c) — Code interpretation [2 marks]

The following code snippet is from the inner loop of a VFI implementation. It solves for the optimal policy at a single grid point.

```
def solve_inner(a, h, z, EV_interp, params):
    w, r, delta_h, A_h, alpha, beta, sigma = params
    best_val = -np.inf
    best_s, best_a_prime = 0, 0
    for s in np.linspace(0, 1, 21):
        h_prime = (1 - delta_h) * h + A_h * (s * h)**alpha
        cash = w * z * h * (1 - s) + (1 + r) * a
        for a_prime in a_grid:
            c = cash - a_prime
            if c <= 0:
                continue
            val = c**(1-sigma)/(1-sigma) + beta * EV_interp(a_prime,
                h_prime)
            if val > best_val:
                best_val = val
                best_s, best_a_prime = s, a_prime
    return best_val, best_s, best_a_prime
```

(i) The code uses a double loop (over `s` and `a_prime`). Explain what `EV_interp(a_prime, h_prime)` computes and why interpolation is needed here.

(ii) The line `if c <= 0: continue` skips certain combinations. Explain the economic and numerical reason for this check.

Solution

(i) `EV_interp(a_prime, h_prime)` evaluates the expected continuation value $E[V(a', h', z') \mid z]$ at the chosen next-period states (a', h'). Interpolation is needed because a' and h' generally do not lie exactly on the discrete grid points used to store V. The function `EV_interp` uses linear (or spline) interpolation between the surrounding grid points to produce a smooth approximation, which allows the optimiser to consider off-grid values of the choice variables rather than being restricted to grid points.

(ii) If a' is too large relative to cash-on-hand, consumption $c = \text{cash} - a'$ becomes negative or zero. The CRRA utility function $c^{1-\sigma}/(1-\sigma)$ is undefined for $c \leq 0$ (and $\ln(c)$ is $-\infty$). By skipping these infeasible combinations, the code enforces the economic constraint that consumption must be strictly positive and avoids numerical errors (NaN or overflow) from evaluating the utility function at invalid inputs.

End of Mock Examination